Operating System Principles

Module – 3

Multi-Processor Scheduling

Multi-Processor Scheduling

- The term **multiprocessor** referred to systems that provided multiple physical processors, where each processor contained one single-core CPU.
- ***Multiprocessor*** now applies to the following system architectures:

  1. Multicore CPUs
  2. Multithreaded cores
  3. NUMA systems
  4. Heterogeneous multiprocessing

Approaches to Multiple-Processor Scheduling

- **Asymmetric multiprocessing (AMP)**

- master server approach

- Only one core accesses the system data structures, reducing the need for data sharing

- The master server becomes a potential bottleneck where overall system performance may be reduced.

Approaches to Multiple-Processor Scheduling

- **Symmetric multiprocessing (SMP)**

- Self scheduling

- Scheduler for each processor examine the ready queue and select a thread to run

- *All threads may be in a common ready queue.*

- *Each processor may have its own private queue of threads.*

Approaches to Multiple-Processor Scheduling

- **Symmetric multiprocessing (SMP)**

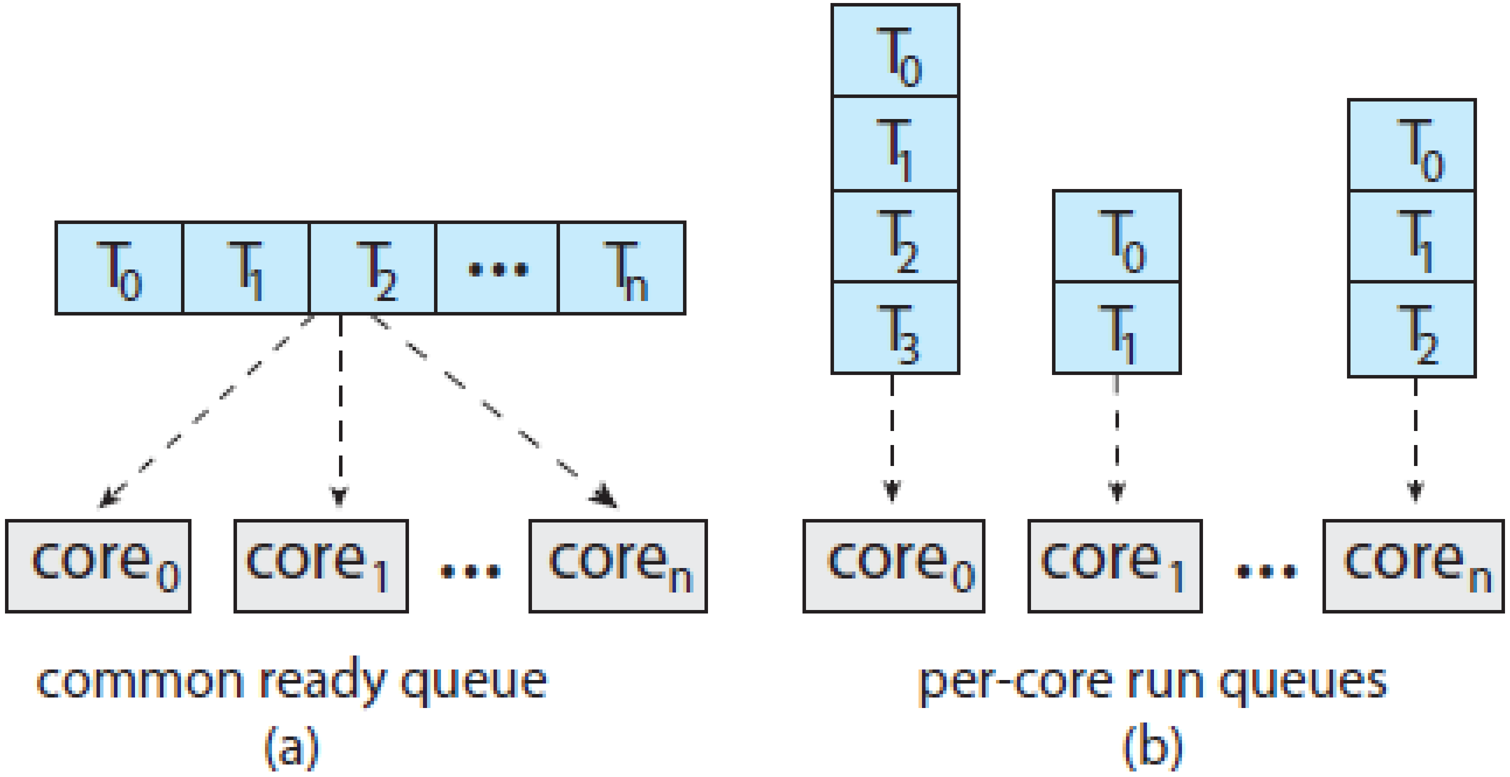

Figure: Organization of ready queues.

Approaches to Multiple-Processor Scheduling

- **Symmetric multiprocessing (SMP)**

- Common ready queue can suffer from race condition

- Locking can be used

- per-core run queues are commonly used – suffer from workloads of varying sizes.

- balancing algorithms can be used to equalize workloads among all processors

Multicore Processors

- SMP systems have allowed several processes to run in parallel by providing multiple physical processors.
- Computer hardware now places multiple computing cores on the same physical chip, resulting in a multicore processor.
- Each core maintains its architectural state and thus appears to the OS to be a separate logical CPU.
- Faster and consume less power

Multicore Processors

- Has scheduling issues
- Memory stall - processors operate at much faster speeds than memory or because of cache miss

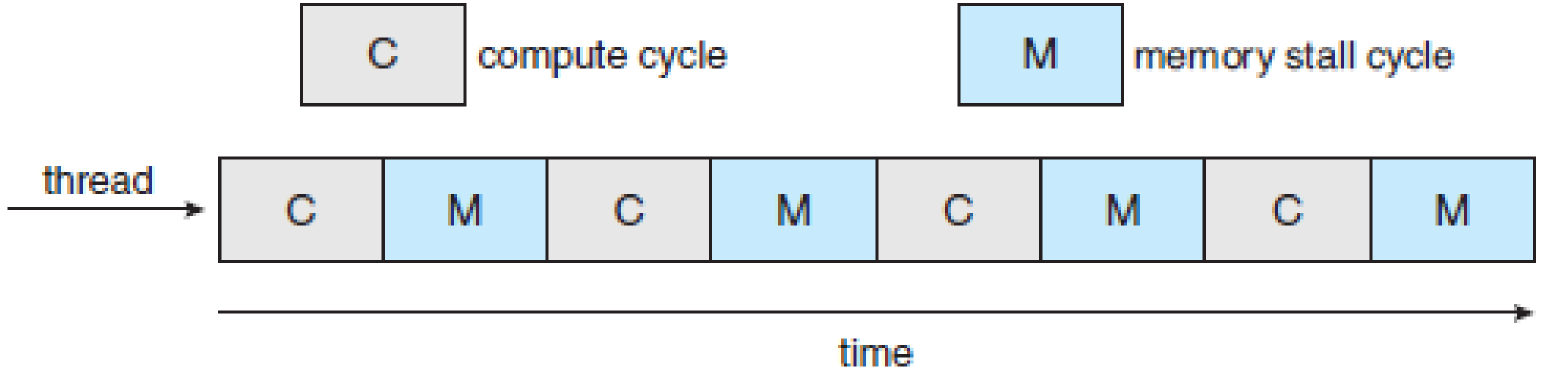

Figure: Memory stall

Multicore Processors

Solution

- multithreaded processing cores in which two (or more) **hardware threads** are assigned to each core.
- the core can switch from one thread to another thread
- In the dual-threaded processing core, the execution of thread 0 and thread 1 are interleaved

Multicore Processors

- Each hardware thread maintains its architectural state, such as instruction pointer and register set, and thus appears as a logical CPU. This technique—known as **chip multithreading (CMT)**

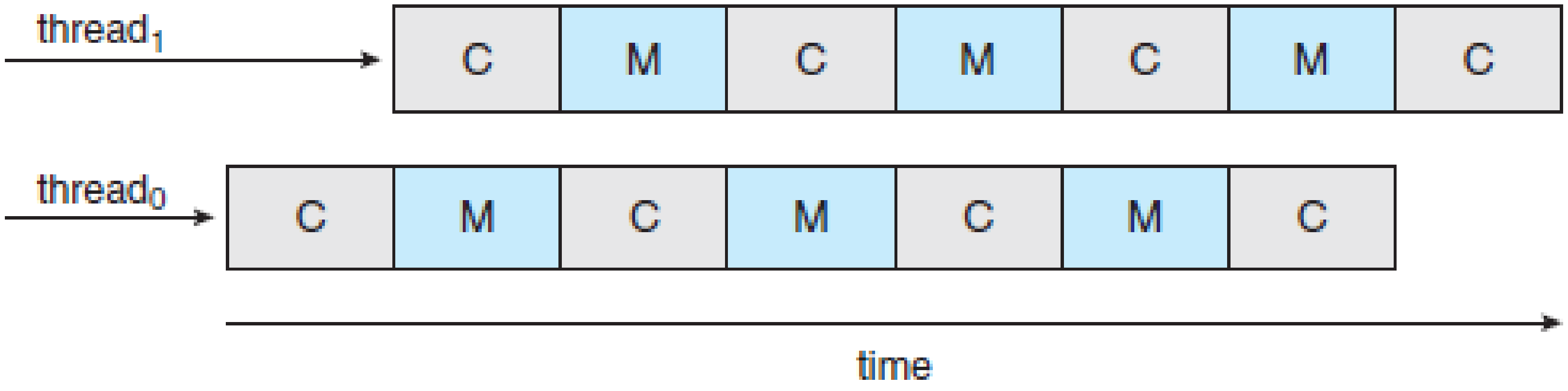

Figure Multithreaded multicore system.

Multicore Processors

- Intel processors use the term **hyper-threading** (also known as **simultaneous multithreading** or SMT) to describe assigning multiple hardware threads to a single processing core.
- For example: Intel processor i7—support two threads per core, while the Oracle Sparc M7 processor supports eight threads per core, with eight cores per processor, thus providing the operating system with 64 logical CPUs.

Multicore Processors

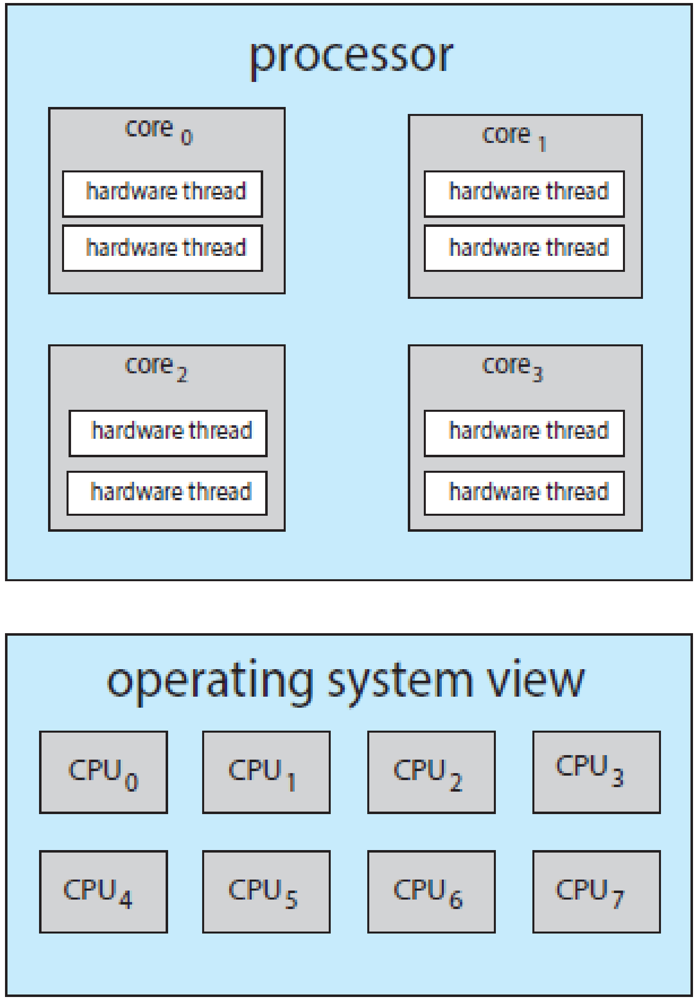

Figure Chip multithreading

Multicore Processors

- Two ways to multithread a processing core: **coarse grained** and **fine-grained.**
- With coarse-grained multithreading, a thread executes on a core until a long-latency event occurs.
- Fine-grained (or interleaved) multithreading switches between threads at a much finer level of granularity

Multicore Processors

- The resources of the physical core must be shared among its hardware threads, and therefore a processing core can only execute one hardware thread at a time.
- Therefore, a multithreaded, multicore processor requires two different levels of scheduling.
- **Scheduling Level 1:** OS as it chooses which software thread to run on each hardware thread (logical CPU).

Multicore Processors

- **Scheduling Level 2:** A second level of scheduling specifies how each core decides which hardware thread to run.

Two strategies to select a hardware thread

- Round-robin algorithm was used to schedule a hardware thread to the processing core.
- Hardware thread is assigned a dynamic ***urgency*** value ranging from 0 to 7,

Multicore Processors

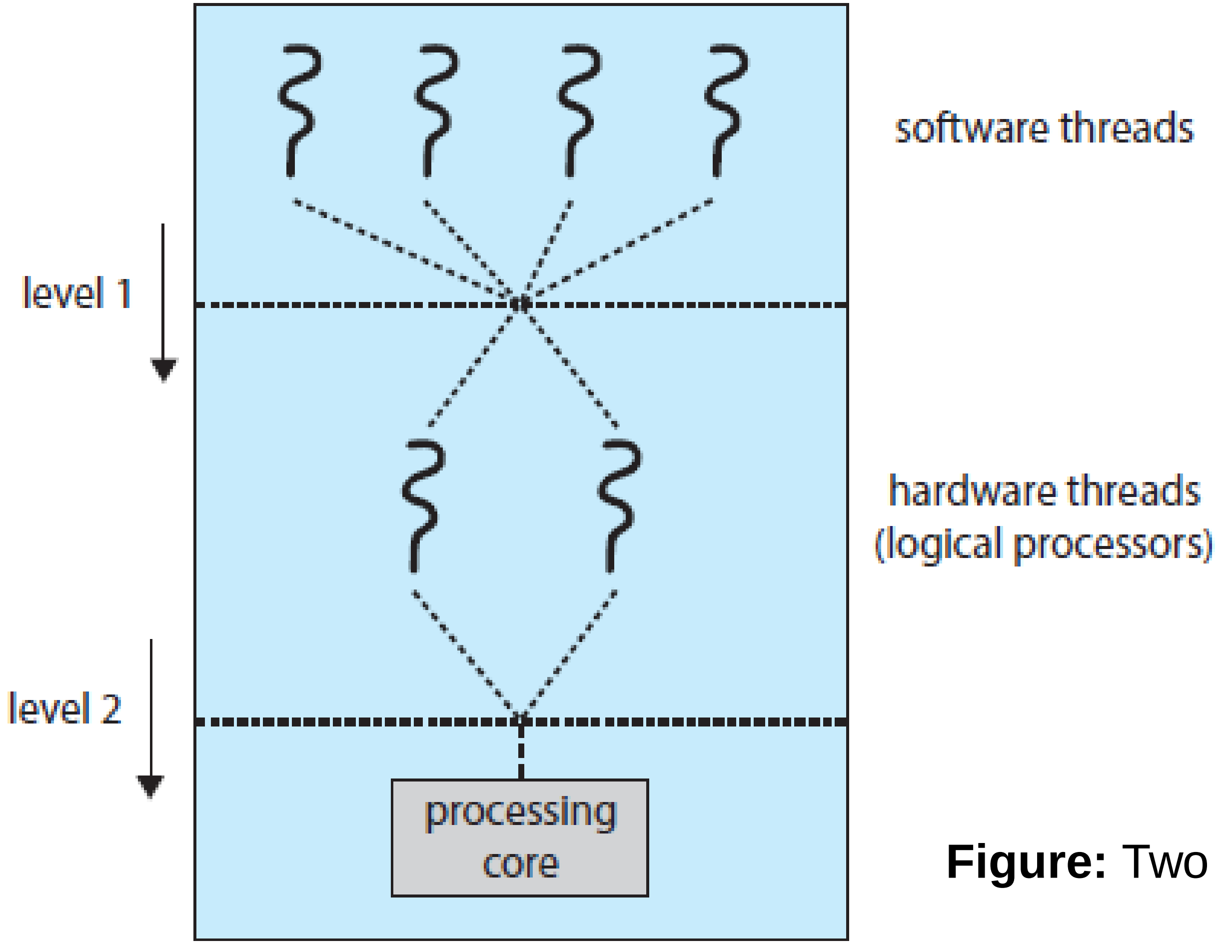

Figure: Two levels of scheduling trigger

Load Balancing

- **Load balancing** attempts to keep the workload evenly distributed across all processors in an SMP system.
- Necessary only on systems where each processor has its own private ready queue

There are two general approaches to load balancing:

1. Push migration
2. Pull migration

Processor Affinity

Consider what happens to cache memory when a thread has been running on a specific processor and migrates to another processor?

Processor affinity — a process has an affinity for the processor on which it is currently running

- per-processor ready queues provide processor affinity for free

Processor Affinity

Soft affinit .

- Here, the operating system will attempt to keep a process on a single processor, but it is possible for a process to migrate between processors during load balancing.

Hard affinit

- Allowing a process to specify a subset of processors on which it can run.

Processor Affinity

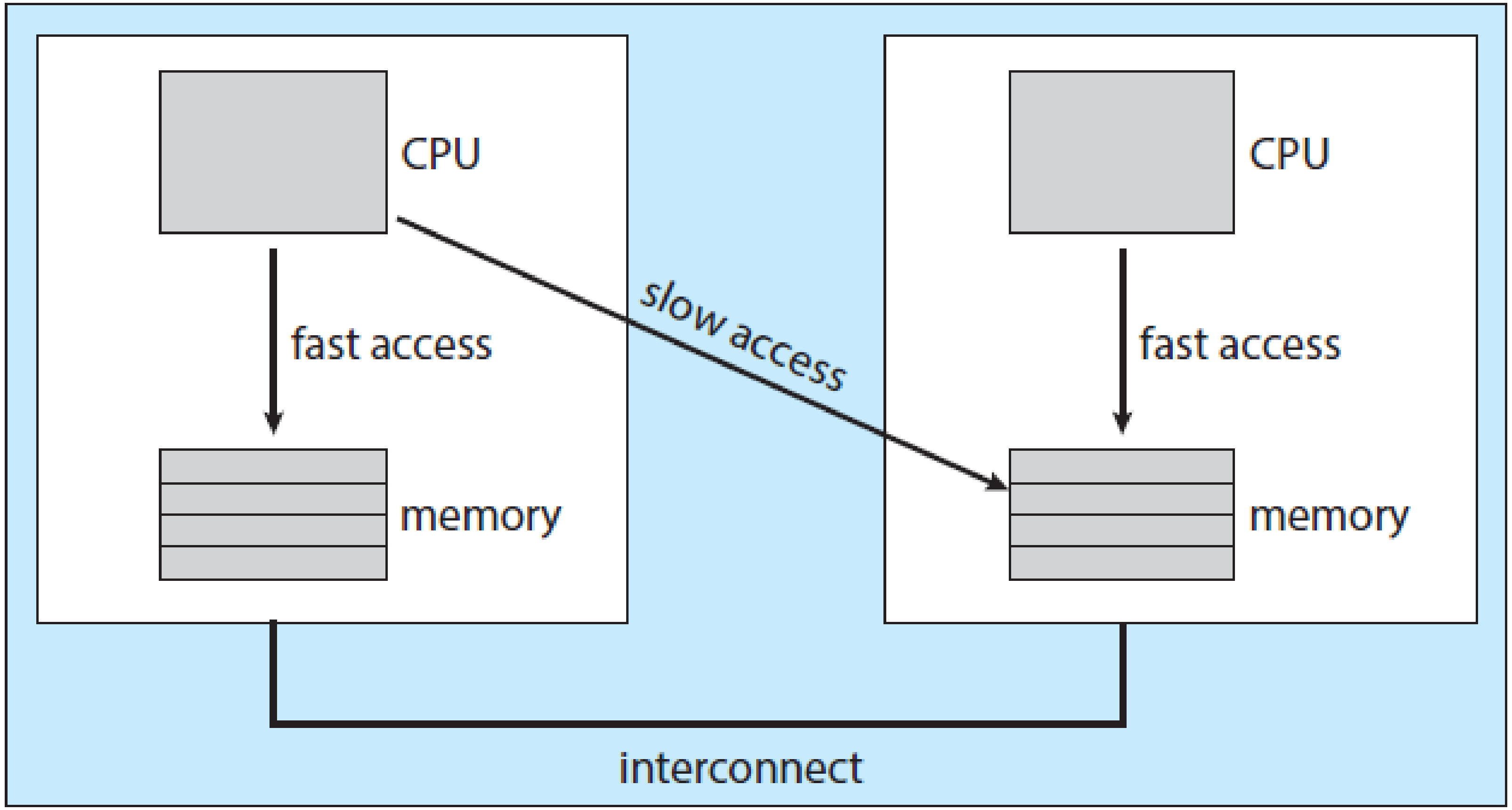

Figure NUMA and CPU scheduling.

Processor Affinity

- The main-memory architecture of a system can affect processor affinity issues.
- In NUMA, a CPU has faster access to its local memory than to memory local to another CPU.
- If the OS's CPU scheduler and memory-placement algorithms are ***NUMA-aware*** and work together, then a thread that has been scheduled onto a particular CPU can be allocated memory closest to where the CPU resides.
- The memory access times may vary based upon load balancing and processor affinity policies

Heterogeneous Multiprocessing - HMP

- HMP is designed to better manage power consumption by assigning tasks to certain cores based upon the specific demands of the task.
- For example: **ARM big LITTLE architecture** where higher-performance ***big*** cores are combined with energy efficient ***LITTLE*** cores.
- When mobile device is in a power-saving mode, energy-intensive big cores can be disabled and the system can rely solely on energy-efficient little cores